

A self-learning, privacy-preserving engine for reading operational reports and mapping them to a standard chart

Design and implementation plan. Written 15 September 2026 for the Open Hospitality project and for any system with the same shape of problem: many source systems, each customised by its users, producing documents full of sensitive detail, that must be turned into rows on a standard chart of accounts — and must keep working as sources change, without a developer hand-coding every one.

Where this paper says "hotel", "night audit", "PMS" and "USALI", read "site", "daily report", "source system" and "your standard chart". Nothing below depends on the industry.

0. What this paper is for

A hotel's front-desk system prints a night audit every morning. There are thousands of such systems, every hotel configures its own charge codes and often its own report layout, and the report carries guest names, card numbers and account balances beside the totals the books need. The job is to read the totals, decide what each line means under the standard chart, post them to a ledger — and do it for a system nobody has seen before, on a computer whose owner has been promised that nothing sensitive leaves it.

The same problem exists for restaurant point-of-sale close-outs, clinic billing summaries, property management statements, franchise royalty reports and payroll registers. This paper describes one engine for all of them: a **ladder** of ever-more-capable readers where the cheapest safe rung answers first; **recipes** — declarative descriptions of layouts — that are learned from a person's confirmation, verified by replay, and reused forever; a **meaning layer** that maps rows to the standard chart from a shared dictionary, the tenant's own decisions and, only when those are silent, a constrained AI; a **community library** that lets the second site on a system pay nothing; and a **security envelope** that makes the privacy promise structural rather than hopeful.

Everything here is buildable with free software. The Open Hospitality desktop edition already has the seed of most layers; the paper says what exists, what is missing, and in what order to build it.

1. Principles

These are not aspirations; each one is a design rule that later sections enforce.

1. **Local first.** Everything that can run on the user's computer runs there. Text extraction, OCR, layout analysis, recipe replay, dictionary lookups, the ledger — none of it needs a network.
2. **One door out.** Exactly one component may make an outbound request: the AI helper, on the user's own account. Nothing else in the product opens a connection, and the bundled tools are launched with networking blocked.

3. **Build, don't filter — where you can.** A question to a model is assembled from a typed object with named fields. There is no path from a database row to a prompt except through that object, so forbidden data is structurally absent.
4. **Filter whole, refuse rather than mask — where you must.** Reading a document nobody wrote a reader for means showing text the product did not write. Then the unit is the page: a page goes whole or is held back whole. If the outbound scan finds anything forbidden, the request is refused and the kind (never the value) is reported. Masking hides bugs; refusal makes them visible.
5. **The model proposes; a person disposes.** Nothing reaches the books, the recipes or the dictionary until a person confirms it, and the decision is recorded in that person's name.
6. **Learn data, not code.** What the system learns is recipes and decisions — declarative records drawn from a fixed vocabulary of layouts and lines. The model never writes code that runs unattended, and a recipe can only pick from shapes the engine already knows how to execute.
7. **Verify by replay.** A recipe is stored only if executing it against the same document reproduces the confirmed rows exactly. A recipe that cannot be replayed is not a recipe.
8. **Drift is loud.** When a recipe finds nothing where it expects something, the read climbs the ladder again and the person is told the source changed. It is never silently re-guessed.
9. **Share by structure.** What is shared with other users is safe because of what it is — a fingerprint, a recipe, a code vote — not because it was scrubbed. The shared schema has no field that could carry a name, an amount, a date range or an identifier.
10. **Measure with a corpus.** Every change to reading or mapping is checked against a growing library of anonymised real documents before it ships. Breadth is a data problem as much as a code problem.

2. The architecture in one picture

Documents enter at the bottom and rows leave at the top; sensitivity decreases as they rise, and so does the amount of the system that is allowed to touch them.

- **L0 Ingress** — files from a folder, an upload, a mailbox or an API; hashed, quarantined, provenance recorded.
- **L1 Extraction** — text and positions from born-digital PDFs; OCR for scans; layout analysis for tables. All local, network-blocked.
- **L2 Recognition** — a fingerprint of the document's kind; a lookup against known recipes, the tenant's and the community's.
- **L3 Reading** — recipe replay: deterministic, offline, verified.
- **L4 Proposal** — for an unknown document: local layout rules, then the AI helper on filtered pages, guided by a skill. Produces a proposed reading for a person to confirm.
- **L5 Meaning** — each row mapped to the standard chart: tenant decisions, then the shared dictionary with votes, then a constrained AI with refusal rules.
- **L6 Posting** — facts, restatement in place, period locks. (Exists; unchanged by this paper.)

- **L7 Learning** — confirmations and corrections become recipes, decisions, drafted skills and corpus samples.
- **L8 Sharing** — the community library, opt-in, structural.
- **Envelope** — the security rules that surround every layer: data classes, storage, keys, network policy, audit.
- **Evaluation** — the corpus and the gates.

Each layer is specified below with what exists today in the Open Hospitality desktop edition, what to build, and how to test it.

3. L0 — Ingress

What it does. Takes a document from wherever it arrives and makes it a first-class record: a content hash (so the same file read twice is read once), the source (folder, upload, mailbox with sender and subject, API), the time, and any hint the source carries (a mailbox rule saying which site a subject line belongs to). It quarantines the file in a system-owned folder until it has been read, then files it as read or as set aside with the reason.

Exists. Folder watch, upload, IMAP collection, hashing, the read / couldn't-read folders, IngestBatch with status and message.

Build.

- Hints as a sidecar record beside the file (subject, sender, the rule's site), read by the ladder and deleted with the file. A hint may fill in what a document does not print; it may never overrule what the document does print — a contradiction sets the file aside with both facts stated.
- CSV and spreadsheet exports as first-class inputs. Most source systems can export; an export has no page scanning and no OCR, and its recipe is a column mapping.
- API pulls, where a source offers one, as the same record type.

Tests. A file seen twice is read once. A hint that contradicts the header is refused with the reason. An export and a PDF of the same report produce the same rows.

4. L1 — Extraction

What it does. Turns a document into words with positions, then into rows and, where possible, tables — without any network.

Three tools, in order of cost.

- **pdfplumber** (MIT) for born-digital PDFs: every word with its x/y. Fast, small, already in use.
- **Tesseract via ocrmypdf** (Apache 2.0) for pages with no text layer: a scanned pack becomes a normal, noisier page. Bundled inside the app folder like the database; launched with networking blocked.

- **A layout model** — Docling (MIT) or Marker — for unknown documents: gives back headings, tables and reading order rather than word soup, so the proposal step (L4) names columns instead of reconstructing a table. Optional download, a few hundred megabytes of models, runs on the PC.

Rules. OCR output and layout output are the most sensitive artefacts in the system (a guest ledger as plain text). They live only in the system folder beside the database, are never written to the user's own folders, are excluded from backups, and are deleted once the document is read or set aside. Row clustering must be the same code the readers use, so a page the model sees is the page the reader would see.

Tests. The three sample packs read identically through pdfplumber and through a rasterise-then-OCR round trip within a stated tolerance. No bundled tool can open a socket (a test that runs it under a deny-all rule and checks it still works).

5. L2 — Recognition: fingerprints

What it does. Decides *what kind* of document this is, without knowing whose it is.

The fingerprint. A normalised tuple of: the page title (first line, case-folded, digits and dates removed), the column headings in order, the count and shape of the header's labelled fields ("Property Code:", "Business Date:"), and the section titles inside a pack. It deliberately excludes every value: no names, no numbers, no dates. Two hotels on the same system and version produce the same fingerprint; a hotel that added a column produces a different one, which is the point.

Lookup order. The tenant's own recipes first; then the community library's; then no match, which sends the document to L4.

Versioning. A source system that changes its layout produces a new fingerprint. Old recipes are kept, marked with the last date they matched, so a hotel that has not upgraded keeps reading.

Exists. Report detection by title and header signature for the three known systems; the property registry (name or code in the header).

Build. The fingerprint function; a fingerprint on every IngestBatch; the lookup.

Tests. The same report from two hotels fingerprints identically; adding a column changes it; a value change (different date, different amounts) does not.

6. L3 — Reading: recipes

What a recipe is. A small declarative record that says how to read one kind of document. It is data, not code. Its vocabulary is closed: it can only name layouts, columns and anchors the engine already knows how to execute.

A recipe has:

- fingerprint — the kind it applies to.

- `site_anchor` — where the site's code or name prints (a labelled field in the header, or "not printed", in which case a hint or the registry supplies it).
- `date_anchor` — where the business date prints and its format.
- `sections` — for a pack, which section titles are read and which are read past.
- `layout` — the line shape: `code description amounts...`, `description (code) amounts...`, `label value`, a column table with named headers, and so on — one of a fixed list.
- `columns` — which column is the night's amount, which the period-to-date, which the prior year; the sign rule (parentheses, trailing minus, a debit/credit pair).
- `statistics` — `label` → metric code (rooms sold, occupancy, ADR...), and which column is the night.
- `overrides` — the tenant's layer on top of a shared base: a renamed section, an extra column, a code printed in a different position.
- `provenance` — who confirmed it, when, how many documents it has read, when it last matched.

Layered recipes. A source system's base recipe is shared; a tenant's overrides are local. Replay applies the base and then the overrides. A tenant's customisation therefore never pollutes the shared record, and a base improvement reaches every tenant that has no override for that part.

Replay. Deterministic, offline, milliseconds. Its output is rows: site, date, code, description, amount, plus statistics.

Verification. A recipe is accepted only if replay against the confirming document reproduces the confirmed rows exactly — every code, description and amount, and the date. If not, nothing is stored and the next document goes up the ladder again.

Drift. Replay that finds no page with its title, no rows, or no date returns "nothing" rather than a guess. The document is re-read by L4, and the person is told the layout changed. A recipe that has drifted is retired when its replacement is confirmed.

Exists. The recipe engine for the AI-read path with a first set of layouts; verification by exact replay; drift returning None.

Build. Widen the layout vocabulary (multi-column statistics, sub-sections, totals lines, debit/credit pairs); the overrides layer; CSV/spreadsheet column-mapping recipes; recipes for the three known systems as shipped files so the hand-coded readers become the reference implementation and then optional.

Tests. Every shipped recipe reproduces the sample corpus exactly; a recipe with an unknown layout name is rejected at load; an override changes only what it names.

7. L4 — Proposal: reading a document nobody has a recipe for

The ladder. Tried in order; the first rung that yields a verified reading wins.

1. **Exact match** — the tenant's own recipe. No model, no cost, every morning after the first.

2. **Community match** — a recipe shared by others for this fingerprint, applied and shown as "read with a layout confirmed by N other sites"; the person may correct it, which makes a tenant override.
3. **Local layout** — the layout model's table plus rules: a date-shaped header, a money-shaped column, a short-code column, parentheses as money out. Proposes rows without any outbound call.
4. **The AI helper** — on pages that passed the holdback (\$15), with the reading guide (skill) for the nearest known system, asked for rows in a fixed JSON shape. Proposes; never posts.

Confirmation. The person sees the proposed rows beside the page, edits what is wrong, and presses "put these in the books". The reading is posted (L6), the recipe is inferred and verified (L3), and, if the tenant opted in, the fingerprint and base recipe are offered to the library (L8).

Skills. A skill is a reading guide for a kind of document: which page holds the figures, which column is the night, that a bracketed amount is money out, where the date prints. It is a Markdown file shipped with the engine, versioned with it, and handed to the model as instructions. Skills carry no site data. After enough confirmed readings on a new system, the model may *draft* a skill from those confirmations; a person reviews and ships it.

Exists. Rungs 1 and 4; skills for three systems; the confirm-then-learn loop.

Build. Rungs 2 and 3; skill drafting from confirmations; the "read with a layout confirmed by N sites" affordance.

Tests. A document with a tenant recipe never reaches the model (a test that fails if the model adapter is called). A community recipe that misreads is correctable and the correction becomes an override, not a shared change.

8. L5 — Meaning: mapping rows to the standard chart

Two kinds of row. Revenue and settlement codes (what the night audit carries) and expenses (what bank and card statements, supplier invoices and payroll carry). Both are mapped the same way.

Precedence. For each code (or merchant, or supplier):

1. **The tenant's own decision**, if one exists — recorded with who decided and when.
2. **The shared dictionary with votes** — "1,214 sites on this system filed PET_FEE under Miscellaneous Income › Pet Fee". Strong agreement pre-confirms; a split shows both readings and asks.
3. **A constrained AI question** — only when the first two are silent. The model chooses from the chart's line list, must give a one-sentence reason, and must decline where the answer turns on tax treatment or on capital-versus-expense. Its answer is a suggestion with a confidence; a person confirms.

Refusal rules are part of the chart. Each chart carries the questions the model must not answer (for USALI: tax lines, capitalisation, inter-company). Another domain edits that list, not the code.

Restatement. Confirming a code that has already been posted re-classifies its facts in place and re-posts every affected day; days inside a locked period are refused by name. (Exists.)

Expenses, specifically.

- **Merchants** from statements: the merchant string is the key; the AI suggests a schedule and line ("Cintas → Rooms › Linen and uniforms"); memory per tenant; votes across tenants.
- **Suppliers** from invoices: the invoice is a document like any other — L1 through L4 apply, the holdback removes bank details (supplier account numbers are on most invoices), a per-supplier recipe is learned.
- **Payroll** from the provider's pay run, mapped by department to the labour schedules.

Exists. Dictionary plus per-tenant decisions for codes; AI suggestion with reason and refusal; merchant memory for card statements; restatement.

Build. The expense side of the chart with its refusal list; merchant and supplier mapping through the same loop; invoices as documents; votes (§10).

Tests. A tenant decision beats the dictionary; the dictionary beats the model; a tax question is declined; a locked period refuses restatement by name.

9. L6 — Posting

Facts from rows; a journal built from facts; corrections as reversals; period locks. This exists and is upstream's engine; the paper changes nothing here except to note two requirements it already meets: posted facts are re-classified in place rather than deleted (the journal references them), and a locked period is a wall, not a warning.

10. L7 and L8 — Learning and sharing

What learns, and from what.

- **Recipes** — from a confirmed reading (§6).
- **Overrides** — from a correction to a community recipe.
- **Decisions** — from a confirmed mapping (§8).
- **Skills** — drafted by the model from a run of confirmations, shipped by a person.
- **The corpus** — from a confirmed reading the tenant opts to donate, anonymised (§17).

What never learns. Model weights (no fine-tuning on customer data); code; anything unconfirmed.

The community library. A hosted service beside the engine (or a signed static file the app downloads) holding, per fingerprint: base recipes with their confirmation counts; per system and code: line votes with counts; skills. What is shared is defined by schema and the schema has no field for a site name, an amount, a date range, a person or an account — so nothing needs scrubbing and nothing can leak by mistake. Sharing is opt-in per tenant, off by default, and the record about to be shared is shown first.

Trust. A recipe's weight is its confirmation count across distinct tenants; a code vote likewise. A tenant may pin a recipe or a line regardless of votes. A shared recipe found to misread is retired by

anyone's correction reaching a threshold of confirmations against it. Nothing in the library is executed as code; a recipe that names an unknown layout is rejected at load.

Exists. Per-tenant decisions and recipes; the memory notes that mirror them for the user to read.

Build. The library service and its client; opt-in and preview; votes; retirement; a local mirror so lookups are offline.

11. Memory — where the knowledge lives

"Memory" is not a model remembering things. It is records in the database, written only when a person confirms, and read every time a document arrives. The model is stateless; the books are the memory. Three tiers:

Tier 1 — the tenant's own knowledge (in the database, in the backups).

- *Decisions*: for this site, on this system, code PET_FEE is filed under Miscellaneous Income > Pet Fee — who decided, when, whether it was the person's own choice or an AI suggestion they accepted, and a note. One row per code per site.
- *Recipes and overrides*: for this fingerprint, how it is read — page title, line layout, which column is the night's amount, where the date prints; when it was confirmed, how many documents it has read, when it last matched; and the site's overrides on a shared base.
- *Merchant and supplier memory*: "Cintas → Rooms > Linen and uniforms", keyed by the merchant string; the same shape as a decision.
- *Site profile*: ownership entity, breakeven, last year's revenue, the mailbox rules ("subject contains ... → this site").
- *Provenance*: every document — when, from where, read or set aside and why; every confirmation in a person's name; every outbound call by kind.

Tier 2 — shipped knowledge (files that come with the engine, versioned with it). The dictionary of common codes per system with confidence and review status; the skills, one reading guide per document kind; the base recipes, the machine-readable form of each skill; the chart's lines and the questions the model must decline.

Tier 3 — the community library (opt-in, structural, mirrored locally). Fingerprint → base recipes with confirmation counts; system and code → line votes; reviewed skills.

Recall is a key, never a search. A document's fingerprint → its recipe. A code → the site's decision, else the dictionary, else the votes. A merchant string → its memory. **Precedence is fixed:** the site's own record beats the shipped dictionary, which beats the community's votes, which beat the model. Learning runs the other way: a model's answer becomes a decision only when a person confirms it, and a decision becomes a vote only when the site opts to share it.

The readable mirror. The same records are written as notes a person can browse — one per site with its decisions and the reports read, one per reading guide — into a folder any notes app opens. It is one-way: the database is the memory, the notes are a window on it, and editing a note changes

nothing. A hand edit that silently changed how figures were filed is the failure the decision records exist to prevent.

12. How the engine knows what to look for

It never reads a whole document and hopes. It looks for anchors, in a fixed order, and the ladder decides how much intelligence each step may spend.

1. **Is this a kind I have seen?** Compute the fingerprint — first line with values stripped, column headings, the labelled header fields, the section titles — and look it up. A match means the recipe says where everything is; nothing is searched.
2. **Whose is it, and which day?** Every report anchors these in its header: a labelled field first ("Property Code:", "Business Date:"), then a date-shaped token in the header window, then the site registry (this site's code or name as it prints). A mailbox hint may fill in what is not printed; it may never overrule what is.
3. **Which pages carry figures, and which carry people?** Section titles decide. Summary, journal, statistics, trial balance are figures. Guest, ledger, aging, in-house, arrivals, departures, folio, cashier, payroll are people — held back before anything else looks at them.
4. **Where is the table?** On a figures page: the header row is the line with the most short capitalised words; the money column is the one whose cells are money-shaped (digits, commas, a decimal, optional parentheses); the code column is short uppercase tokens beside descriptions; a date column is date-shaped. Rows are clustered by vertical position with the same clustering the readers use, so a model or a rule sees the table as printed.
5. **Does it add up?** Totals are anchors too: the rows must sum to the printed total, and a night audit nets to zero across charges and settlements. A reading that fails the arithmetic is shown as an anomaly, never posted — this is what catches an OCR misread.
6. **Only then, the model** — only for an unknown kind, only on the figures pages, only with the nearest skill as its instructions, and only to name columns and rows, never to invent them. Its answer is verified by replay before it is remembered.

So "what to look for" is mostly not a question the model answers. It is a fixed list of anchors, and the model is the last resort for the one case — a new layout — where the anchors need naming.

13. Photographs of receipts

A photograph is a scanned page with worse geometry. It goes down the same ladder with two extra rungs at the bottom, and it is a natural fit for expenses (§8): a receipt is a supplier document, and what comes out is a merchant, a date, a total and line items to map to the chart's expense lines.

How it arrives. The same doors as any document: emailed to the mailbox the engine collects from (the phone's camera, then "share" to mail), or saved to the drop folder (a synced folder on the phone). A dedicated "receipts" address with a rule that files everything from the owner's phone as expenses is a small addition.

What happens to it — on the PC, nothing outbound.

1. *Straighten it.* Detect the paper's edges, correct the perspective, deskew, lift the contrast (OpenCV, free). A phone photo of a receipt is trapezoidal, shadowed and low-contrast; correcting that is most of OCR accuracy.
2. *OCR with Tesseract*, as for any scan. Receipts are single-column and narrow, which OCR does well; the failure mode is digits ("5" read as "S", "0" as "O"), which the arithmetic check catches.
3. *Receipt anchors.* Merchant: the largest text at the top. Date: the first date-shaped token. Total: the largest amount, or the one beside "TOTAL". Tax: beside "TAX". Line items: money-shaped rows between the header and the total. A masked card number is expected; a full one is refused by the holdback and the owner is told why.
4. *Arithmetic.* Lines plus tax must equal the total; if not, the photo is shown beside the reading for the person to fix — the same confirm step as everything else.
5. *Meaning.* The merchant goes through the expense loop: memory first, then votes, then the constrained model with the capital-versus-expense refusal — the question that matters most for receipts, and precisely the one the model is told to decline.
6. *Memory.* A confirmed merchant → line is remembered; a merchant whose receipts always look alike gets a recipe, so the second receipt from the same shop reads without a model.

Two limits, stated. A crumpled or dark photograph may not OCR at all; the engine says "couldn't read this one — take it again in better light" rather than guess. And receipts carry more personal detail than operational reports (a cardholder name, the last four digits, a loyalty number): the holdback's rules apply, and the local-model tier is the right setting for an owner who photographs many receipts.

Cost. About a week on top of the OCR fallback in the plan: the straightening step, the receipt anchors, the "take it again" feedback. The mapping side is shared with statements and invoices, so it comes almost free once the expense phase exists.

14. The security envelope

Data classes. Every artefact is one of:

- **Class A — sensitive source:** the document itself, OCR text, layout output, page text. Stays in the system folder; excluded from backups; deleted after reading; never shown to a model except through the holdback.
- **Class B — books:** rows, facts, journal, decisions, recipes with overrides. In the database; in backups (encrypted, keyed by the user's recovery code); never leaves except as the user's own exports.
- **Class C — shareable by structure:** fingerprints, base recipes, code votes, skills. May go to the library with opt-in.
- **Class D — secrets:** provider keys, mail passwords. The operating system's credential store; never in the database, a file, a log, a backup or an error message.

Network policy. One outbound path (the AI helper) plus the library client and the update check, each to a named host, each recorded. Bundled tools run with networking blocked; the app makes no other

connection.

The holdback (§15) guards the one path that carries free text.

Audit. Every outbound request is recorded: kind, size, cost, outcome. Every confirmation is recorded in a person's name. Every refusal names the kind of thing found, never the value; no error may carry the request.

Backups carry Class B and D-wrapped keys only; never Class A.

Supply chain. Pinned versions and checksums for every bundled tool; no self-update; the update check reads one published file and sends nothing.

15. The PII holdback, specified

Two paths, two protections.

Constructed questions (classify a code): a typed object — system, code, printed description, dates seen, amounts, candidate lines. No other field can exist. The serialized request is scanned anyway, and refused if the scan finds anything.

Filtered documents (read an unknown report): before the scan, a page is held back for *what it is* — its title names a section about people or accounts (guest, ledger, aging, in-house, arrivals, departures, no-show, reservations, cashier, folio, registration, payroll, employee, staff, tax-exempt, pre-paid, deposit, credit limit, direct-bill detail, company, travel agent, member), or it lists people in mixed case line after line. Then the scan runs on the page text and refuses on:

- a Luhn-valid card number on one line, or split across whitespace, tabs, hyphens and line breaks in card-like groups (4-4-4-4, 4-6-5);
- nine or more bare digits in a row;
- a Social Security number as printed, or with its hyphenated groups split across a break;
- a name as a source system prints one (DOE, JANE), including with the break after the comma;
- any of the tenant's own staff names, any case;
- an email address; a phone number.

What it does not do, stated: it does not join every digit across every space (that would hold back statistics pages), it cannot recognise a lone mixed-case name in prose on an otherwise clean page (that residual is why pages are judged by what they are first), it never masks, it never logs a value.

Tests are the specification: every shape above is refused and the refusal never quotes the value; built questions are never refused; real summary and statistics rows pass; on the sample pack the totals survive and the guest pages do not.

OCR. OCR'd text is the hardest input: numbers split across lines, letters misread as digits. The cross-break rules exist for it; the corpus must include OCR'd samples; and the local-model tier (§16) is the right setting for a site whose packs are scans.

16. The AI usage contract

- **Whose account.** The user's — a provider they chose, a key they pasted, held in the credential store. The product holds no key and takes no margin.
- **Tiers.** Cloud provider through the one door; or a model on the PC (Ollama, LM Studio) through the same port, for a site that wants zero outbound.
- **Questions.** Two shapes only: classify a code (constructed) and read pages (filtered). Both ask for a fixed JSON reply and nothing else.
- **Refusals.** The model is told when to decline; the reply schema has a `decline_reason`; a decline is shown as an answer, not an error.
- **Caps.** A monthly dollar cap when prices are known and a count cap always; enforced before the call.
- **Record.** Every call: kind, tokens, cost, outcome. Never the content.
- **Errors.** Never carry the request.

17. Evaluation: the corpus and the gates

The corpus. Anonymised real documents: words and positions kept, every value replaced — names by placeholders, digits by other digits of the same shape, amounts by amounts that preserve the totals' arithmetic. Each sample carries its confirmed rows as the expected answer. Grown by opt-in donation at confirmation time, and by the maintainers' own testers.

Metrics.

- Read accuracy: rows exactly right per document; per system; per rung of the ladder.
- Held-back rate: pages held back per document, and whether the totals survived.
- Mapping agreement: dictionary and model suggestions versus confirmed decisions.
- Cost: model calls and dollars per site per month; time to first correct read on a new system.
- Drift detection: layout changes caught before a wrong number posted.

Gates. No change to L1–L5 ships if any corpus sample reads worse; no change to the holdback ships if any forbidden shape passes; no shipped recipe or skill without a corpus sample that exercises it.

18. Step-by-step implementation plan

Each phase ends with its tests green, the corpus unchanged or improved, and — for a desktop deployment — the packaged end-to-end walk passing.

Phase 0 — Foundations (1 week). Fingerprint function and storage; the data-class rules made explicit in paths and backups (Class A never backed up, deleted after reading); network-blocked launch of bundled tools; the audit record for outbound calls extended to the library and update check. *Exit:* fingerprints on every batch; a test proves a bundled tool cannot open a socket.

Phase 1 — Extraction (1 week). Bundle Tesseract/ocrmypdf; OCR fallback for text-less pages; the layout model as an optional download; OCR artefacts in the system folder only. *Exit:* the sample packs read through a rasterise-and-OCR round trip; a scanned sample joins the corpus.

Phase 2 — Recipes as the readers (2 weeks). Widen the layout vocabulary; recipe overrides; CSV and spreadsheet column-mapping recipes; shipped recipes for the known systems, verified against the corpus; the hand-coded readers kept as the reference until the recipes match them, then made optional. *Exit:* every corpus sample reads by recipe replay with the same rows as the hand-coded readers.

Phase 3 — The ladder (1 week). Rungs 2 and 3; the "read with a layout confirmed by N sites" affordance; corrections as overrides; drift climbing the ladder with the person told. *Exit:* a document with a tenant recipe never reaches a model (test); a changed layout is reported, not guessed.

Phase 4 — Meaning for expenses (3 weeks). The chart's expense side with its refusal list; merchant mapping from statements through the confirm-remember loop; invoices as documents with the holdback (bank details); payroll by department; photographs of receipts (§13): straightening, receipt anchors, the take-it-again feedback. *Exit:* a statement's merchants map with reasons; an invoice with an account number is held back; a capital-versus-expense question is declined.

Phase 5 — The community library (2 weeks). The shared schema (structural: no field for a value); the service or signed file; the client with a local mirror; opt-in with preview; votes and confirmation counts; retirement by correction. *Exit:* a second tenant on a system reads its first document with no model call; the schema is shown to be unable to carry a name or an amount.

Phase 6 — Skills drafted from confirmations (1 week). The model drafts a reading guide from a run of confirmed readings; a review step; shipped with the engine. *Exit:* a drafted skill for a new system, reviewed, makes rung 4 read that system correctly on the first attempt in the corpus.

Phase 7 — The corpus and the gates (1 week, then ongoing). Anonymiser; donation at confirmation; the metrics; the gates in CI. *Exit:* a deliberately broken recipe fails the gate.

Phase 8 — Adapting to another domain (1 week per domain). Swap the chart (its lines and its refusal list); add the domain's source types and skills; seed the corpus. Nothing in L0–L4 or L7–L8 changes. *Exit:* the domain's first three sources read and map through the same ladder.

Total: about twelve weeks of engineering for the hotel case, most of which is engine work that belongs with the upstream project, and about a week per additional domain thereafter.

19. Adopting this design in another system

To reuse the engine for a different kind of report:

1. Define the **standard chart**: its lines, and the questions a model must decline.
2. Define the **document kinds**: what a source prints, where its site and date anchors are, which sections carry figures and which carry people.

3. Write the **section deny-list** for the holdback: the titles of pages that are about people or accounts in your domain.
4. Provide **three real samples per source**, anonymised, with confirmed rows — the first corpus.
5. Write one **skill** per source kind in plain language: which page, which column, the sign rule, the date.
6. Point the **community library** at your own instance, or run without one.

Everything else — extraction, fingerprints, recipes, the ladder, the holdback, the AI contract, the audit, the gates — is the same code.

20. Risks, and what answers them

- *A model reads a clean-looking page that carries one guest's name in prose.* The section deny-list and the density rule catch the pages names live on; the residual is stated, the local-model tier removes it entirely, and the corpus measures it.
- *A shared recipe misreads for a site with a custom layout.* Fingerprints include the column set, so a custom layout is a different kind; corrections become overrides, never shared changes.
- *A source changes its layout overnight.* Replay returns nothing; the ladder re-reads; the person is told; the old recipe is retired only when the new one is confirmed.
- *A user pastes a key that then leaks in a log.* Keys live in the credential store; errors never carry requests; every outbound call is recorded by kind, never content.
- *The library is poisoned with a wrong vote.* Weight is distinct-tenant confirmation count; a tenant can pin; corrections retire entries; nothing shared is executed.
- *OCR turns a number into a different number.* Totals lines and arithmetic checks in recipes catch a misread that breaks the sum; the anomaly is shown, not posted.
- *The corpus itself leaks.* It holds positions and placeholders; the anonymiser's tests prove no original value survives.

21. Glossary

- **Chart** — the standard set of lines figures are filed under (USALI for hotels).
- **Corpus** — the anonymised sample library used to gate changes.
- **Fingerprint** — the value-free signature of a document kind.
- **Holdback** — the rules that keep a page or a request from leaving.
- **Ladder** — the ordered set of readers, cheapest and safest first.
- **Override** — a tenant's layer on a shared recipe.
- **Recipe** — a declarative, verified description of how to read one document kind.
- **Replay** — executing a recipe; deterministic and offline.
- **Skill** — a plain-language reading guide handed to the model.
- **Tenant** — one site or group whose books are kept together.

